



COS 214 Practical 5

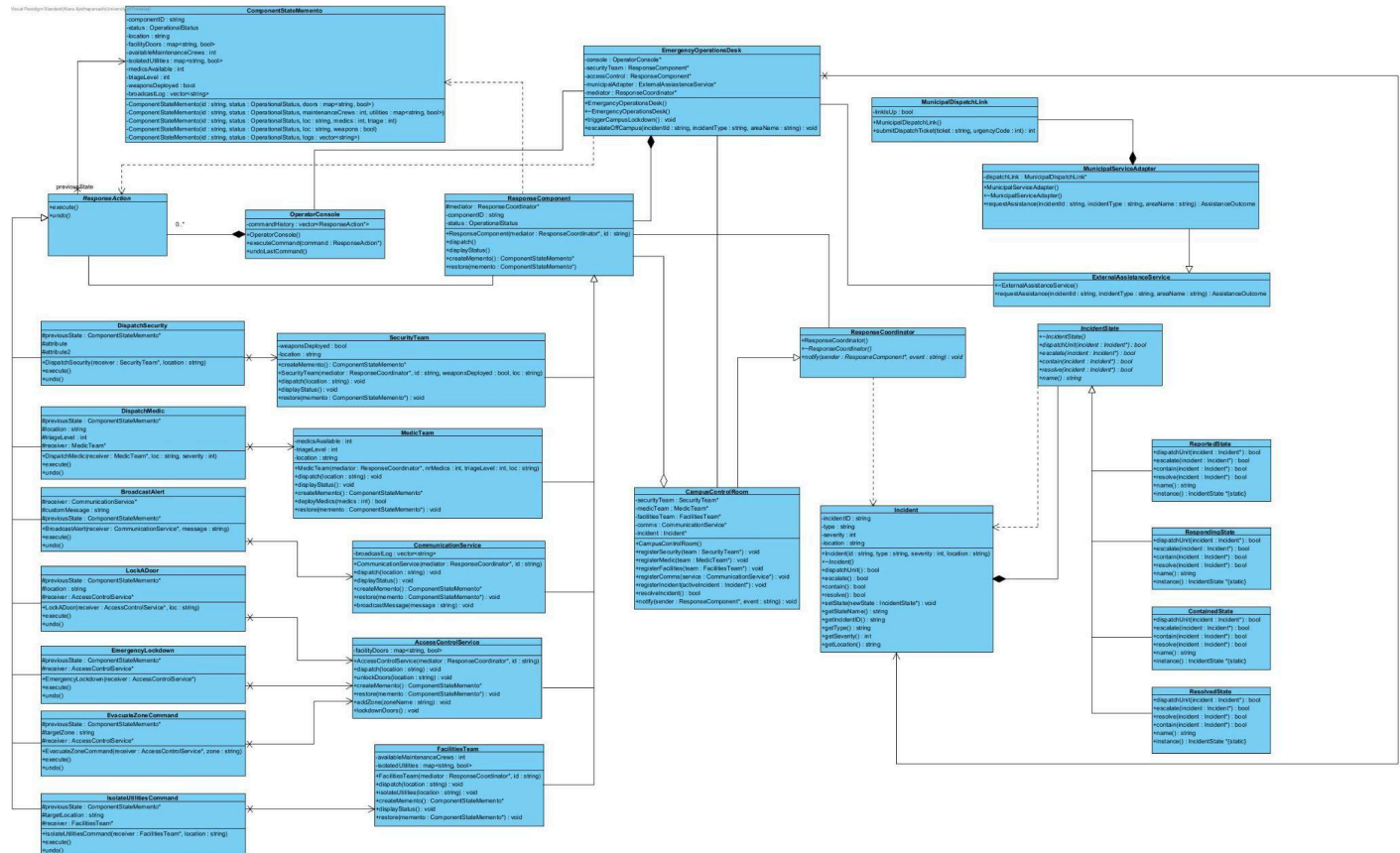
29.09.2026

Group members

Carla Erasmus (u21450481)

Kiara Ajodhaparsadh (u25395344)

Vimbai Chivunga (u25136608)



Identification of the GoF participants for Command, Mediator, Adapter and Facade

Command

Command	ResponseAction
ConcreteCommand	DispatchSecurity, DispatchMedic, EmergencyLockdown, LockADoor, BroadcastAlert, EvacuateZone
Invoker	OperatorConsole
Receiver	SecurityTeam, MedicTeam, FacilitiesControlService and CommunicationService
Client	EmergencyOperationsDesk

Mediator

Mediator	ResponseCoordinator
ConcreteMediator	CampusControlRoom
Colleague	ResponseComponent
ConcreteColleague	SecurityTeam, MedicTeam, FacilitiesTeam, CommunicationService, AccessControlService

Adapter

Target	ExternalAssistanceService
Adapter	MunicipalServiceAdapter
Adaptee	MunicipalDispatchLink
Client	EmergencyOperationsDesk

Facade

Facade	EmergencyOperationsDesk
Subsystem	OperatorConsole, AccessControlService, CommunicationService, CampusContolRoom, Incident, IncidentLogbook, ExternalAssistanceService
Client	main

Two additional GoF patterns selected by your team and a short justification for each additional design pattern

Memento

We chose the Memento pattern to allow emergency response units (like a SecurityTeam or AccessControlService) to save and safely restore their exact operational states—such as location, available personnel, and equipment deployment. This enables the operator console to cleanly undo emergency commands without breaking object encapsulation or exposing the units' private variables.

This approach is far more robust than a simple "cancel" command. For example, if doors 1 and 2 are originally locked while doors 3 and 4 are unlocked, triggering an EmergencyLockdown will lock all of them. If we only relied on a basic cancel function, undoing the lockdown might just unlock every door indiscriminately. By using a Memento, the system captures the exact previous configuration, ensuring that when the lockdown is lifted, doors 1 and 2 remain securely locked while doors 3 and 4 are unlocked.

Originator	ResponseComponent (and SecurityTeam, MedicTeam, FacilitiesTeam, CommunicationService, AccessControlService)
Memento	ComponentStateMemento
Caretaker	OperatorConsole (and concrete commands like EmergencyLockdown)

State

The scenario requires our design to be able to register incidents and change their operational status. We decided State would be best suited to fulfill this requirement since a resolved incident should not be able to accept a dispatch, an unconfirmed report shouldn't trigger a campus-wide alert and escalation is only legal from an active state. Without the STATE design pattern this would require huge if chains through the commands and the facade. The state transition is what the mediator broadcasts on. A command executes, the incident changes state, the state change is the event the control room coordinates on.

Context	Incident
ConcreteState	ReportedState, RespondingState, ContainedState, ResolvedState
State	IncidentState

Important associations, inheritance relationships, multiplicities and ownership decisions

Incident owns its current state.

OperatorConsole owns actions once submitted.

ResponseAction never owns its receiver.

MunicipalServiceAdapter owns its MunicipalDispatchLink, created in its constructor and deleted in its destructor.

Facade owns nothing

Incident (1) owns its current IncidentState (1).

OperatorConsole (1) owns ResponseAction (0..*) commands once submitted.

ResponseAction (1) never owns its receiver (1).

MunicipalServiceAdapter (1) owns its MunicipalDispatchLink (1), created in its constructor and deleted in its destructor.

Short explanation of where the six patterns interact rather than operate independently (design rationale)

The six patterns interact with each other to solve a problem one couldn't alone. The strongest interaction is between Memento and Command. When the OperatorConsole executes a ResponseAction, the command can capture a ComponentStateMemento from the target ResponseComponent before altering it. If the operator calls undoLastCommand(), the command uses this memento to perfectly rewind the unit's internal variables (like reverting a SecurityTeam's location and weapons status) without violating encapsulation.

The Facade manages these checkpoints, ensuring big operations either finish completely or reset cleanly. If an outside city computer sends back a confusing error, an Adapter translates that code into a clear failure signal, which tells the Facade to restore the checkpoint.

Deciding what is allowed is split between Command and State. A Command asks to change an incident, but the State decides if that change follows campus safety rules. This cleanly separates asking for an action from checking if it is legal.

When a state change is approved, it alerts the Mediator (the dispatch room), which keeps campus teams aligned around actual milestones rather than random alerts. The Facade also runs its own work through Commands so everything stays logged, and it hides the city Adapter behind a safe escalateOffCampus step. Finally, when the Mediator alerts teams, it creates new Commands of its own. This turns a single operator's decision into clear, tracked steps across the whole system rather than untraced background activity.

Task 3 : Scenarios

Story 1: Severe fire in ENG-LAB-2

The operator manages everything through **EmergencyOperationsDesk (Facade)** instead of touching subsystems directly.

They first try to call in outside help too early - the incident is still in the Reported state, so it refuses (**State**), and the **Adapter** is never reached.

Next they trigger a lockdown. The Facade submits three commands to **OperatorConsole (Command)**. Dispatching security also makes the mediator lock a door and send a warning on security's behalf (**Mediator**), and moves the incident to the Responding state (**State**).

The operator then undoes a badly-worded broadcast — the command's saved memento restores the comms log (**Memento**).

With the incident now Responding, they escalate again. This time it's allowed, so the request goes through **MunicipalServiceAdapter**, which translates it into the legacy format and back (**Adapter**). The municipal service accepts.

Facade drives it, Command carries every action, Mediator coordinates the fan-out, State gates both escalation attempts, Memento undoes the bad broadcast, Adapter reaches the external service.

Story 2: Minor gas leak in CHEM-BLDG

Facilities isolate the gas (**Command**), and the mediator tells comms (**Mediator**). Security and medics are dispatched next, each command triggering the mediator again and security's dispatch also moves the incident to Responding (**State**).

An accidental campus-wide evacuation alert is sent, then undone - the memento restores the comms log (**Memento**).

A full lockdown follows, taking the incident to Contained (**Mediator + State**), and once things are safe, it's resolved (**State**) - completing Reported → Responding → Contained → Resolved.

Finally, someone tries to dispatch medics to the resolved incident. It's refused (**State**), but the medic team had already acted, so the operator undoes it (**Command + Memento**) to clean up.

We intentionally did not use the no Facade or Adapter patterns in this scenario to demonstrate that the subsystems work fine without them.

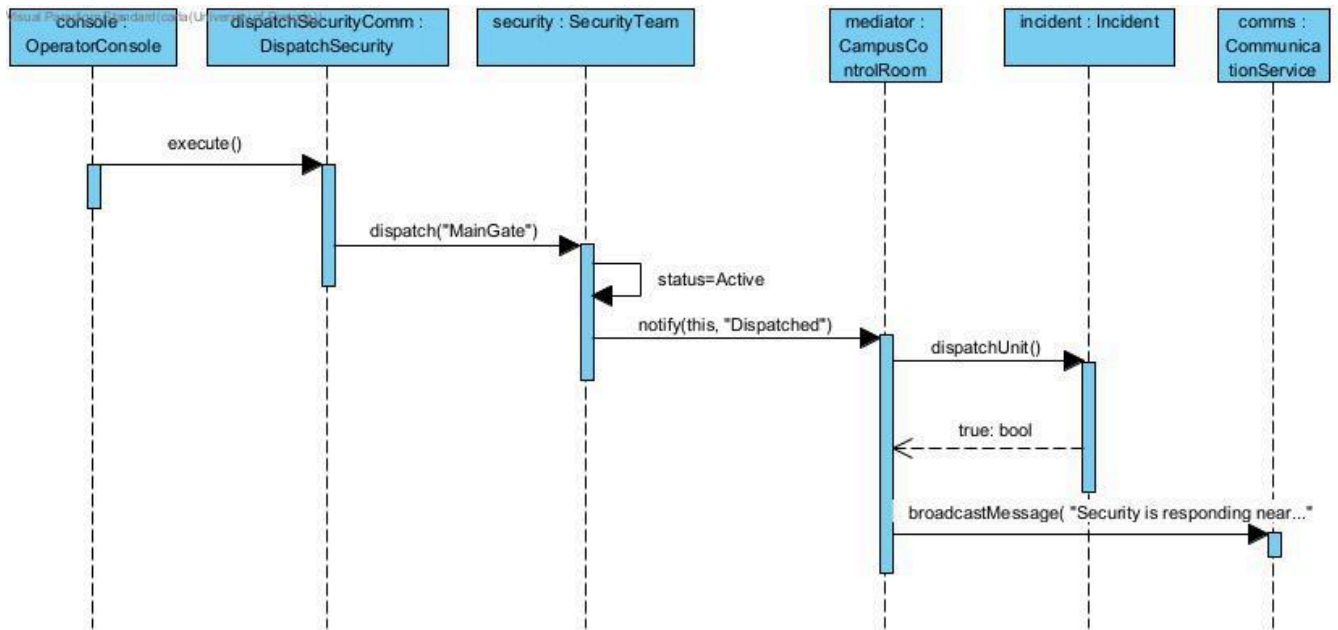
Task 4: Diagrams

Sequence diagram 1

UML sequence diagram showing a substantial runtime interaction that includes Command plus at least one other required pattern.

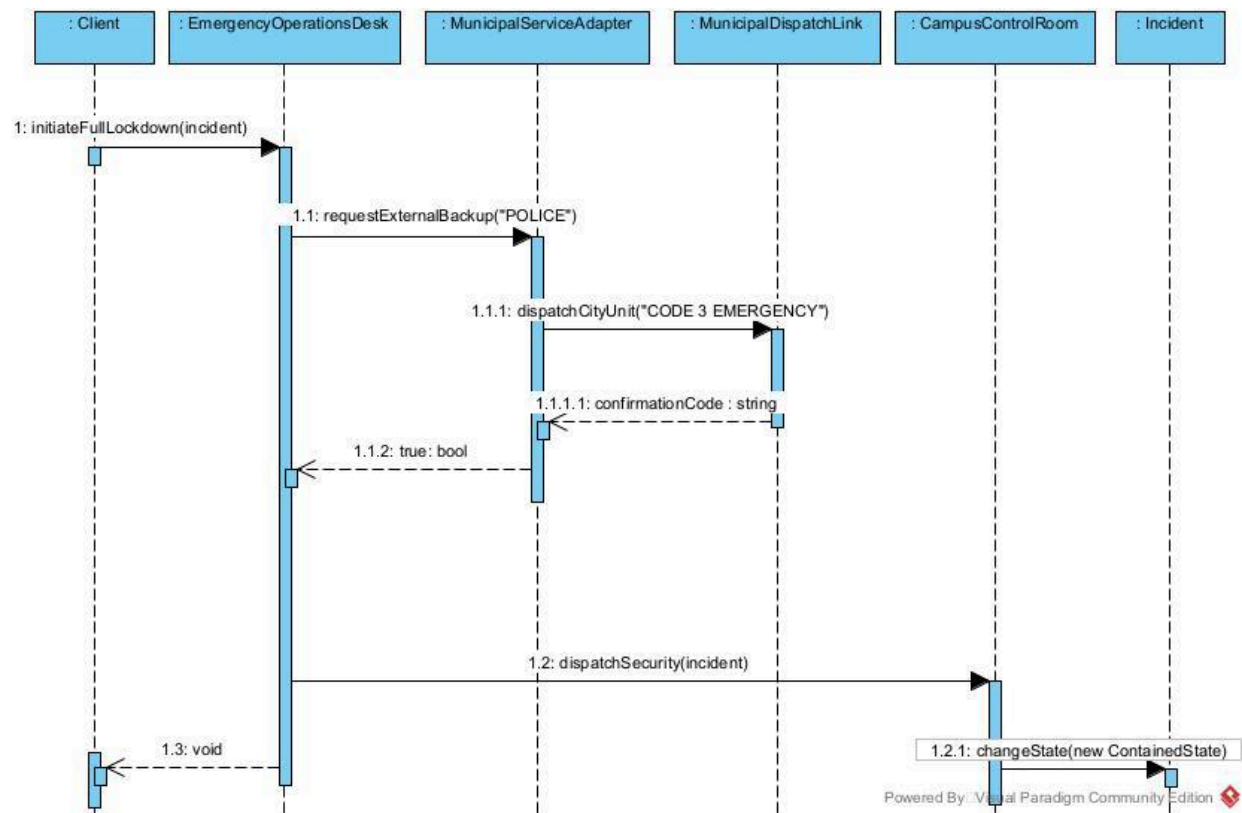
This sequence diagram illustrates the runtime interaction between the Command and Mediator patterns. It begins with the OperatorConsole executing a DispatchSecurity command, which directly triggers the SecurityTeam's dispatch method. After updating its

internal status, the SecurityTeam alerts the CampusControlRoom mediator. The mediator first verifies the action's legality with the current Incident state. Upon receiving a true boolean return, the mediator completes the workflow by coordinating with the CommunicationService to broadcast a campus-wide warning message.



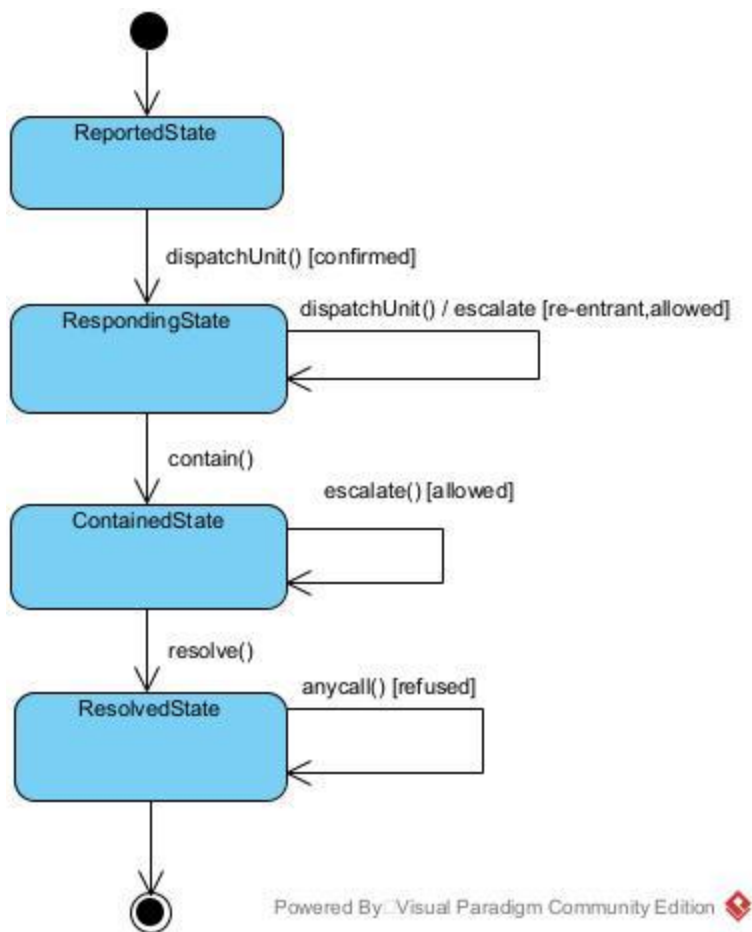
Sequence diagram 2

This sequence diagram illustrates how the system handles a lockdown when a client invokes `initiateFullLockdown(incident)`. The `EmergencyOperationsDesk` acts as a Facade, serving as the single entry point to hide complex subsystem interactions from the client. To communicate with external services, it relies on the `MunicipalServiceAdapter` (Adapter), which translates internal backup requests into calls compatible with the legacy `MunicipalDispatchLink`. Once external dispatch is confirmed, the desk coordinates with `CampusControlRoom` to dispatch local security and update the `Incident` state to `ContainedState`.



State diagram

This state machine diagram illustrates the formal lifecycle and state transition constraints of an Incident with the CampusGuard platform.



Task 5: engineering quality

Ownership policies

CampusControlRoom does not own its colleagues or the Incident it coordinates - they're built independently and registered afterward, since a component needs a ResponseCoordinator* before it can exist, and the mediator has no need to outlive or destroy them.

OperatorConsole owns its command history and deletes every command on destruction; each command owns the one memento it captured and deletes it in its own destructor.

MunicipalServiceAdapter owns its MunicipalDispatchLink, with copying disabled since duplicating that ownership has no sensible meaning.

EmergencyOperationsDesk, as the composition root, owns the mediator, console, every component, the incident, and the adapter, and destroys them in reverse order of construction.

Valgrind evidence

The program was executed inside the Docker container using Valgrind (valgrind --leak-check=full --show-leak-kinds=all ./campusguard). As shown in the summary above, all heap blocks were freed upon program exit with 0 bytes leaked across 34 allocations and 0 errors detected. This confirms proper dynamic memory management, virtual destructors across polymorphic hierarchies, and clean ownership policies.

```

Problems Output Debug Console Terminal Ports
vimbai_chivunga@Vimbai:/mnt/c/Users/Vimbai_Chivunga/Desktop/YEAR 2 SEMESTER 2/COS 214/Practical 5/COS214PRAC5-CampusGuard$ sudo docker run --rm -it campusgua
rd-app valgrind --leak-check=full --show-leak-kinds=all ./campusguard
[sudo: authenticate] Password:
==1== Memcheck, a memory error detector
==1== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==1== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==1== Command: ./campusguard
==1==

--- Dispatch security: fans out via the Mediator AND advances the Incident (Reported -> Responding) ---
[OpertorConsole] Executing operator command...
[Incident INC-001] Reported -> Responding (first unit dispatched).
[CampusControlRoom] Security dispatched -> coordinating facilities and comms.
[CampusControlRoom] Broadcast logged.
Incident is now: Responding

--- Undo that dispatch (Command + Memento) - note this rolls back SecurityTeam, NOT the incident's lifecycle ---
[OpertorConsole] Initiating system undo sequence...
[SecurityTeam: Sec-Alpha ] Reverted to previous state successfully. Location, status and weaponsDeployed updated.
=== [SECURITY: Sec-Alpha] ===
> Location: Standby/Base
> Status: Idle
> Weapons: Secured
=====

--- Re-dispatch security and medics, then lock down all doors (Responding -> Contained) ---
[OpertorConsole] Executing operator command...
[Incident INC-001] Additional unit dispatched while Responding.
[CampusControlRoom] Security dispatched -> coordinating facilities and comms.
[CampusControlRoom] Broadcast logged.
[OpertorConsole] Executing operator command...
[Incident INC-001] Additional unit dispatched while Responding.
[CampusControlRoom] Medical team dispatched -> notifying campus.
[CampusControlRoom] Broadcast logged.
Incident is now: Responding

--- Resolve the incident (contained -> Resolved) ---
[Incident INC-001] Cannot be resolved - incident is currently Responding.
Incident is now: Responding

--- State pattern's failure case: dispatching to an already-resolved incident is refused ---
[OpertorConsole] Executing operator command...
[Incident INC-001] Additional unit dispatched while Responding.
[CampusControlRoom] Security dispatched -> coordinating facilities and comms.
[CampusControlRoom] Broadcast logged.
==1==
==1== HEAP SUMMARY:
==1==    in use at exit: 0 bytes in 0 blocks
==1== total heap usage: 34 allocs, 34 frees, 77,063 bytes allocated
==1==
==1== All heap blocks were freed -- no leaks are possible
==1==
==1== For lists of detected and suppressed errors, rerun with: -s
==1== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
vimbai_chivunga@Vimbai:/mnt/c/Users/Vimbai_Chivunga/Desktop/YEAR 2 SEMESTER 2/COS 214/Practical 5/COS214PRAC5-CampusGuard$

```

GDB investigation

Screenshot 1: Successful compilation of the final CampusGuard executable

```

lsCOS214/P5/CampusGuard-fixed/CampusGuard-fixed$ make
g++ -std=c++11 -Wall -Wextra -g -c CommunicationService.cpp -o CommunicationService.o
g++ -std=c++11 -Wall -Wextra -g -c ResponseComponent.cpp -o ResponseComponent.o
g++ -std=c++11 -Wall -Wextra -g -c MunicipalServiceAdapter.cpp -o MunicipalServiceAdapter.o
g++ -std=c++11 -Wall -Wextra -g -c MunicipalDispatchLink.cpp -o MunicipalDispatchLink.o
g++ -std=c++11 -Wall -Wextra -g -c MedicTeam.cpp -o MedicTeam.o
g++ -std=c++11 -Wall -Wextra -g -c BroadcastAlert.cpp -o BroadcastAlert.o
g++ -std=c++11 -Wall -Wextra -g -c IsolateUtilitiesCommand.cpp -o IsolateUtilitiesCommand.o
g++ -std=c++11 -Wall -Wextra -g -c EmergencyLockdown.cpp -o EmergencyLockdown.o
g++ -std=c++11 -Wall -Wextra -g -c IncidentState.cpp -o IncidentState.o
g++ -std=c++11 -Wall -Wextra -g -c SecurityTeam.cpp -o SecurityTeam.o
g++ -std=c++11 -Wall -Wextra -g -c EmergencyOperationsDesk.cpp -o EmergencyOperationsDesk.o
g++ -std=c++11 -Wall -Wextra -g -c LockADoor.cpp -o LockADoor.o
g++ -std=c++11 -Wall -Wextra -g -c CampusControlRoom.cpp -o CampusControlRoom.o
g++ -std=c++11 -Wall -Wextra -g -c AccessControlService.cpp -o AccessControlService.o
g++ -std=c++11 -Wall -Wextra -g -c DispatchMedic.cpp -o DispatchMedic.o
g++ -std=c++11 -Wall -Wextra -g -c OperatorConsole.cpp -o OperatorConsole.o
g++ -std=c++11 -Wall -Wextra -g -c ResponseCoordinator.cpp -o ResponseCoordinator.o
g++ -std=c++11 -Wall -Wextra -g -c ComponentStateMemento.cpp -o ComponentStateMemento.o
g++ -std=c++11 -Wall -Wextra -g -c main.cpp -o main.o
g++ -std=c++11 -Wall -Wextra -g -c FacilitiesTeam.cpp -o FacilitiesTeam.o
g++ -std=c++11 -Wall -Wextra -g -c Incident.cpp -o Incident.o
g++ -std=c++11 -Wall -Wextra -g -c DispatchSecurity.cpp -o DispatchSecurity.o
g++ -std=c++11 -Wall -Wextra -g -o campusguard CommunicationService.o ResponseComponent.o MunicipalServiceAdapter.o MunicipalDispatchLink.o MedicTeam.o BroadcastAlert.o IsolateUtilitiesCommand.o EmergencyLockdown.o IncidentState.o SecurityTeam.o EmergencyOperationsDesk.o LockADoor.o CampusControlRoom.o AccessControlService.o DispatchMedic.o OperatorConsole.o ResponseCoordinator.o ComponentStateMemento.o main.o FacilitiesTeam.o Incident.o DispatchSecurity.o
kiara_wsl@DESKTOP-SVBAVPN: /mnt/c/Users/HP/OneDrive - University of Pretoria/Documents/semester 2, year 2/COS 214/Practicals/2023/P5/CampusGuard-fixed/CampusGuard-fixed$ ./campusguard

```

GDB investigation of a bug:

```

Segmentation fault (core dumped)
● carla@DESKTOP-3F3M4DM:~/2026s2/COS214/COS214PRAC5-CampusGuard$ gdb ./campusguard
GNU gdb (Ubuntu 12.1-0ubuntu1~22.04.2) 12.1
Copyright (C) 2022 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./campusguard...
(gdb) run
Starting program: /home/carla/2026s2/COS214/COS214PRAC5-CampusGuard/campusguard
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
[OperatorConsole] Initiating system undo sequence...

Program received signal SIGSEGV, Segmentation fault.
OperatorConsole::undoLastCommand (this=0x7fffffff9e0) at OperatorConsole.cpp:36
36      ResponseAction *prevCommand = commandHistory.back();
(gdb) bt
#0  OperatorConsole::undoLastCommand (this=0x7fffffff9e0) at OperatorConsole.cpp:36
#1  0x0000555555555686ab in main () at main.cpp:209
(gdb) frame 0
#0  OperatorConsole::undoLastCommand (this=0x7fffffff9e0) at OperatorConsole.cpp:36
36      ResponseAction *prevCommand = commandHistory.back();
(gdb) list
31          return;
32      }*/
33
34      std::cout << "[OperatorConsole] Initiating system undo sequence...\n";
35
36      ResponseAction *prevCommand = commandHistory.back();
37      prevCommand->undo();
38
39      // remove command from history.
40      // if we want to implement redo later, wi do not need to delete it here

```

```

(gdb) print prevCommand
$1 = (ResponseAction *) 0x7fffffff9e0
(gdb) print *prevCommand
$2 = {_vptr.ResponseAction = 0x0}
(gdb) frame 1
#1 0x000055555555686ab in main () at main.cpp:209
209         c.undoLastCommand();
(gdb) list
204
205         // Passing null to your invoker/operator console:
206         c.executeCommand(evacCommand);
207         delete evacCommand;
208
209         c.undoLastCommand();
210         return 0;
211     }
(gdb) quit
A debugging session is active.

        Inferior 1 [process 717186] will be killed.

Quit anyway? (y or n) y
❖ carla@DESKTOP-3F3M4DM:~/2026s2/COS214/COS214PRAC5-CampusGuard$

```

I was getting this frustrating segmentation fault when trying to undo an action, and I couldn't understand why, so I used GDB to see what was happening.

First, I used the run command to recreate the crash, which immediately halted the program with a SIGSEGV error. To find the exact source of the crash, I typed bt to pull up the backtrace, which showed that the issue originated in main.cpp but the actual failure happened inside OperatorConsole.cpp.

I switched to that specific context using frame 0 and typed list to view the surrounding code. This revealed the program was dying on line 36 right at ResponseAction *prevCommand = commandHistory.back();. To see exactly what the computer was trying to read, I used print *prevCommand, which returned a null virtual pointer (0x0).

That showed me that the commandHistory vector was completely empty, meaning .back() was trying to grab a command that didn't exist. To fix it, I simply added an if (commandHistory.empty()) check at the start of the function, so now it gracefully prints "Error: no commands to undo" and safely returns instead of crashing the whole system.

Git/Github

How was the work divided?

Carla implemented Command and Mediator. She did the README and GDB investigation. She made main.cpp more interactive and made one sequence diagram.

Kiara implemented Mediator and State. She made the Makefile and implemented both end to end scenarios. She made the UML.

Vimbai implemented the Adapter and Facade. She made the dockerfile and she made one sequence diagram and an additional behavioural diagram.

How were changes integrated?

We had a main branch, which was protected by a branch ruleset. Each team member made feature branches and committed their changes to their branch. Once a team member was happy with their changes, a pull request would be made. Only then would their branch be merged to main. This helped as we avoided merge conflicts and ensured that the shared codebase always compiled cleanly before any new features were introduced to the main branch. Pull requests were opened and merged without formal review; the equivalent discussion happened between us directly rather than in pull request comments.

<https://github.com/kiaraajodhaparsadh/COS214PRAC5-CampusGuard>

